

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 24 settembre 2009 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. Si consideri l'espressione regolare R seguente, di alfabeto $\{a, b\}$:

$$R = (a b)^* a \mid a b a$$

Si risponda alle domande seguenti:

- (a) Si dica se l'espressione R è ambigua e, in caso positivo, s'indichi quali sono le stringhe ambigue.
 - (b) Si ricavi in modo sistematico dall'espressione R un automa nondeterministico N con mosse spontanee che ne accetti il linguaggio. Si eliminino le mosse spontanee di tale automa. Se necessario, si elimini il nondeterminismo dall'automata risultante. Se necessario, si minimizzi il numero degli stati dell'automata ottenuto, utilizzando un procedimento sistematico.
 - (c) (facoltativa) Con un procedimento sistematico, si ricavi direttamente dall'espressione R un automa deterministico D che ne accetti il linguaggio e, se necessario, se ne minimizzi il numero di stati.
-

2. È data l'espressione regolare E seguente, estesa con l'intersezione:

$$E = \left(\left((a^+ \mid b^+) b \right)^+ \cap \left((a^* \mid b^*) a a (a^* \mid b^*) \right) \right)^*$$

Si risponda alle domande seguenti:

- (a) Si costruisca, facendo comprendere il procedimento seguito, un automa finito A , deterministico oppure non, che riconosca il linguaggio regolare L definito da E .
 - (b) (facoltativa) Si discuta il metodo migliore per costruire un'espressione regolare R del linguaggio L , con i soli operatori classici (unione concatenamento stella e croce).
-

Soluzione

(a) Un procedimento è il seguente:

i. Costruire gli automi di E_1 e E_2 , applicando uno dei metodi noti.

$$E = \left(\underbrace{((a^+ | b^+) b)^+}_{E_1} \cap \underbrace{((a^* | b^*) aa (a^* | b^*))}_{E_2} \right)^*$$

ii. Costruire la macchina del prodotto cartesiano

iii. Da essa costruire la macchina della stella, che è quella richiesta.

(b) Due vie sono possibili:

- Dall'automa prodotto come soluzione della precedente domanda si ricava l'espressione regolare mediante il metodo di eliminazione.
- Si ragiona sui linguaggi L_1 e L_2 per capire com'è fatta l'intersezione. L_1 contiene tutte e sole le stringhe di lunghezza ≥ 2 terminanti con b . L_2 contiene tutte e sole le stringhe di lunghezza ≥ 2 contenenti il digramma aa . $L_1 \cap L_2$ contiene quindi tutte e sole le stringhe di lunghezza ≥ 3 terminanti con b e contenenti il digramma aa , ossia è definito dall'espressione

$$E_3 = (a^* | b^*) aa (a^* | b^*) b$$

Per ottenere l'espressione regolare di L , basta applicare la stella a E_3 .

2 Grammatiche libere e automi a pila 20%

1. Si consideri il linguaggio libero L seguente, di alfabeto $\{a, b\}$:

$$L = \{ a^h b^s a^k \mid s \geq h + k \geq 0 \}$$

Ecco alcune stringhe di esempio:

ε b $a b$ $b a$ $a b b a$ $a b b b a$ $a a b b b a$

e di controesempio:

a $a a b$ $a b b a a$

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica G , ambigua o no, che genera il linguaggio L .
 - (b) (facoltativa) Se necessario, si raffini l'analisi e si scriva una grammatica G' , non ambigua, che genera il linguaggio L .
-

Soluzione

(a) È facile esprimere L come concatenamento:

$$L = \{ a^h b^m \mid m \geq h \geq 0 \} \cdot \{ b^n a^k \mid n \geq k \geq 0 \}$$

dove si avrà $s = m + n \geq 0$ e, dato che $m \geq h$ e $n \geq k$, ne segue che $s \geq h + k$. I fattori $a^h b^m$ ($m \geq h \geq 0$) e $b^n a^k$ ($n \geq k \geq 0$) sono due linguaggi liberi ben noti. Dunque si ha:

$$G \left\{ \begin{array}{l} S \rightarrow X Y \\ X \rightarrow a X b \\ X \rightarrow X b \\ X \rightarrow \varepsilon \\ Y \rightarrow b Y a \\ Y \rightarrow b Y \\ Y \rightarrow \varepsilon \end{array} \right.$$

Si osservi che $L(Y)$ è l'immagine riflessa di $L(X)$.

Beninteso, la scomposizione di L data sopra presenta ambiguità di concatenamento, dunque la grammatica G è ambigua. Per esempio, si ha $a b^2 = a b^2 \cdot \varepsilon = a b \cdot b$.

(b) Per avere una grammatica G' non ambigua, basta raffinare la scomposizione data sopra eliminando l'ambiguità di concatenamento, come per esempio:

$$L = \{ a^h b^h \mid h \geq 0 \} \cdot \{ b^n a^k \mid n \geq k \geq 0 \}$$

dove si ha $s = h + n \geq 0$ e dato che $n \geq k$, ne segue $s \geq h + k$, da cui:

$$G' \left\{ \begin{array}{l} S \rightarrow X Y \\ X \rightarrow a X b \\ X \rightarrow \varepsilon \\ Y \rightarrow b Y a \\ Y \rightarrow b Y \\ Y \rightarrow \varepsilon \end{array} \right.$$

che non è ambigua. In alternativa, si può ribaltare il ruolo di X e Y .

2. Il linguaggio considerato si ispira a quelli delle applicazioni CAD geometriche. Il linguaggio permette di definire una o più rette; ogni retta è specificata da una coppia di punti attraversati dalla retta. Un punto è dato da una coppia x, y di coordinate (numeri reali).

Esempio:

```
LINE line identifier  
myLine2 {1.5, 10} {-2.5, 4, 1} [ red ]  
  
LINE myLine4 {0, 0} {1, -1.5}  
  
LINE parallel {5, 0} {6, -1.5} [ black ] [ dashed ]
```

Una retta può essere ulteriormente specificata con lo stile che può essere **solid** o **dashed**, e il colore **red** oppure **black**, ma tali attributi possono mancare.

Si risponda alle domande seguenti:

- (a) Si scriva una grammatica non ambigua nella forma estesa EBNF per detto linguaggio.
- (b) Si estenda la grammatica precedente per permettere che uno o entrambi i punti che determinano una retta possano essere dati mediante l'intersezione di due rette. Tali rette, usate per intersecare, sono prive di attributi di stile e colore.

Esempio, dove soltanto il secondo punto è dato mediante intersezione:

```
LINE anotherLine {1.5, 10}  
{ LINE Line47 {0, 0} {1, -1.5}  $\cap$  LINE myLine4 {0.3, 0.4} {2.1, -1.5} }  
[black] [dashed]
```

Si noti che anche le rette intersecanti possono essere definite mediante ulteriori rette intersecanti.

3 Analisi sintattica e parsificatori 20%

1. È data la grammatica G seguente, di alfabeto $\{a, b\}$ (assioma Z):

$$G \left\{ \begin{array}{l} Z \rightarrow a Y Z X \mid b X \\ Y \rightarrow X Y \mid b \\ X \rightarrow b X \mid a \mid Z \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si disegnino gli automi corrispondenti alle regole della grammatica G e, mostrando su di essi gli insiemi guida rilevanti di lunghezza $k = 1$, si dica se la grammatica è $LL(1)$.
- (b) In caso negativo, considerando gli insiemi guida rilevanti di lunghezza $k = 2$, si dica se la grammatica G è $LL(2)$.
- (c) (facoltativa) In caso negativo, indichi se la grammatica è $LL(k)$, per qualche $k \geq 3$.

Soluzione

- (a) La grammatica non è $LL(1)$.
- (b) La grammatica non è neppure $LL(2)$.
- (c) La grammatica è ambigua, quindi non è $LL(k)$ per alcun k .

2. Si consideri la grammatica G seguente, di alfabeto $\{a, b, c\}$ (assioma S):

$$G \left\{ \begin{array}{lcl} S & \rightarrow & X S \\ S & \rightarrow & X \\ X & \rightarrow & a X b \\ X & \rightarrow & c \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si dica se la grammatica G è $LR(0)$, giustificando la risposta.
- (b) Se necessario, si dica se la grammatica G è $LR(1)$, motivando la risposta.
- (c) (facoltativa) Mediante l'algoritmo di Earley, si analizzi la stringa $a c b \in L(G)$.

Soluzione

- (a) La grammatica non è $LR(0)$, perché il linguaggio ha prefissi.
- (b) La grammatica è $LR(1)$ (grafo pilota senza conflitti).
- (c) da fare ... (è breve)

4 Traduzione e analisi semantica 20%

1. Si consideri la grammatica G seguente, che modella un frammento di un linguaggio aritmetico con due operatori ternari infissi rappresentati dai simboli ‘#’ ‘;’ e ‘?’
‘?’, rispettivamente.

$$G \left\{ \begin{array}{l} E \rightarrow E \text{ ‘#’ } E \text{ ‘;’ } E \\ E \rightarrow E \text{ ‘#’ } E \text{ ‘?’ } E \\ E \rightarrow i \end{array} \right.$$

Per esempio, appartengono al linguaggio le tre stringhe seguenti:

$$\begin{array}{l} i \text{ # } i \text{ ; } i \\ i \text{ # } i \text{ # } i \text{ ? } i \text{ ; } i \\ i \text{ # } i \text{ ; } i \text{ # } i \text{ ? } i \end{array}$$

Si risponda alle domande seguenti:

- (a) Si definisca una grammatica di traduzione che permetta di ottenere stringhe dove l'ordine in cui compaiono i due simboli negli operatori è invertito. Quindi, le tre stringhe di esempio sopra riportate verrebbero rispettivamente tradotte così:

sorgente	traduzione
$i \text{ # } i \text{ ; } i$	$i \text{ ; } i \text{ # } i$
$i \text{ # } i \text{ # } i \text{ ? } i \text{ ; } i$	$i \text{ ; } i \text{ ? } i \text{ # } i \text{ # } i$
$i \text{ # } i \text{ ; } i \text{ # } i \text{ ? } i$	$i \text{ ; } i \text{ # } i \text{ ? } i \text{ # } i$

- (b) Si discuta la possibilità di effettuare la traduzione arricchendo opportunamente un parsificatore deterministico ascendente per la grammatica G sorgente.

Soluzione

- (a) Basta scambiare, nella grammatica pozzo G_p , i due simboli che compongono l'operatore

$$G_p \left\{ \begin{array}{l} E \rightarrow E \text{ ';} E \text{ '#'} E \mid E \text{ '?'} E \text{ '#'} E \\ E \rightarrow i \end{array} \right.$$

- (b) Si vede facilmente che, dovendo scambiare il primo e il secondo simbolo degli operatori, il parsificatore arricchito dovrebbe emettere i caratteri ';' e '?' prima di effettuare la riduzione e quindi non avendo ancora riconosciuto di quale operatore si tratta. È comunque possibile calcolare la traduzione in modo deterministico ascendente definendo un'opportuna grammatica di traduzione in forma postfissa che permetta effettuare azioni di scrittura nelle sole mosse di riduzione.

2. Si consideri un frammento nello stile del linguaggio Pascal, contenente dichiarazioni di matrici a una o più dimensioni e istruzioni di assegnamento agli elementi delle matrici, come segue:

$A : \text{array}[1 : 5]; \quad C : \text{array}[1 : 5, 1 : 4]; \quad \dots;$ – declarative part
 $B[i, j] := 2; \quad B[h, k] := 2; \quad \dots;$ – statements
 $B[h] := 2; \quad A[h, k] := 2;$ – due errori di dimensione

Si suppone che le variabili intere $i, j, h, k, \dots$ siano state dichiarate altrove.

La sintassi del frammento è data (omettendo molti segni di interpunzione):

sintassi

1	$S \rightarrow \text{decls} ; \text{stats}$
2	$\text{decls} \rightarrow \text{decl} \text{ decls}$
3	$\text{decls} \rightarrow \text{decl}$
4	$\text{decl} \rightarrow \text{name array} [\text{intervals}]$
5	$\text{intervals} \rightarrow \text{num1 num2 intervals}$
6	$\text{intervals} \rightarrow \text{num1 num2}$
7	$\text{stats} \rightarrow \text{name} [\text{indices}] := \text{const} ; \text{stats}$
8	$\text{stats} \rightarrow \text{name} [\text{indices}] := \text{const} ;$
9	$\text{indices} \rightarrow \text{int_var} , \text{indices}$
10	$\text{indices} \rightarrow \text{int_var}$

Si deve progettare una grammatica con attributi che calcola un diagnostico per ogni errore di dimensione incontrato. Nell'esempio gli attributi calcolati sono:

error 1: B has dimension 2;

error 2: A has dimension 1;

Si risponda alle domande seguenti:

- Si scrivano negli appositi spazi le regole semantiche (nella tabella di seguito). Per ogni attributo semantico utilizzato, si spieghi a che cosa serve e se è sinistro o destro.
- Si verifichi se la grammatica con attributi progettata è a una scansione (one-sweep).

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato

sintassi	funzioni semantiche da scrivere
1 $S_0 \rightarrow \text{decls}_1 ; \text{stats}_2$	
2 $\text{decls}_0 \rightarrow \text{decl}_1 \text{ decls}_2$	
3 $\text{decls}_0 \rightarrow \text{decl}_1$	
4 $\text{decl}_0 \rightarrow \text{name}_1 \text{ array } [\text{intervals}_2]$	
5 $\text{intervals}_0 \rightarrow \text{num1}_1 \text{ num2}_2 \text{ intervals}_3$	
6 $\text{intervals}_0 \rightarrow \text{num1}_1 \text{ num2}_2$	
7 $\text{stats}_0 \rightarrow \text{name}_1 [\text{indices}_2] := \text{const}_3 ; \text{stats}_4$	
8 $\text{stats}_0 \rightarrow \text{name}_1 [\text{indices}_2] := \text{const}_3 ;$	
9 $\text{indices}_0 \rightarrow \text{int_var}_1 , \text{indices}_2$	
10 $\text{indices}_0 \rightarrow \text{int_var}_1$	

Soluzione

- (a) da fare ...
- (b) da fare ...